

Scalability group tracker with the label of Saturation Metrics.

Please see the docs-hub on Capacity Planning for more information on the forecasting tool we develop and use.

Triage Duties

The triage duties docs have been moved to the Observability team's docs-hub.

SECTION: engineering/infrastructure-platforms/production-engineering/observability/cost.md

Elastic Cloud Costs (Snowflake)

Where to Find the Views

You can find the view here.

Access Requirements

To view the dashboard, you need access to Snowflake as a SNOWFLAKEANALYST. Follow these steps to gain access:

1. Create an Access Request (AR): Use the provided template to submit your AR.
2. Create a Merge Request (MR): Once your AR is approved, create an MR to grant yourself permissions. Use the template and instructions provided.

After gaining access, you will be able to view and execute the dashboard.

How the Views Are Built

The views for Elastic Cloud are constructed using SQL queries (worksheets in Snowflake). The main query retrieves daily cost data for Elastic Cloud services by SKU from the billing table for the gitlab-logs-prod deployment. It organizes this data by date, SKU, and daily cost to help track and analyze expenses. These visualizations are reusable and can be integrated into other Snowflake dashboards.

What the Views Display

1. Breakdown By SKU (Daily): Shows daily ECU expenditure by SKU.
2. ECU Spend (Monthly): Displays total ECU expenditure by SKU for each month.
3. Breakdown by Data Transfer: Illustrates daily ECU spent on data transfer, including in, out, and between nodes.
4. ECU Burndown (Daily): Provides a burndown chart of daily ECU expenditure, starting from the total ECU available as of February 16th 2024 (when we started recording this data in Snowflake).
5. Estimated ECU Exhaustion: Shows a table with the estimated ECU exhaustion date, calculated by dividing the remaining balance by the daily average cost (The daily average cost is found by first adding up the daily costs for each day and then averaging these totals).

GCP Cost (Looker Studio)

Where to Find the Views
You can find the view here.

How the Views Are Built

1. GKE Cluster Cost and Usage Calculation

This query assesses the cost of the GKE cluster and tracks CPU/memory usage for each namespace using data from the `gkeusagemetering.gkeclusterresourceusage` table in BigQuery and GCP billing export. Costs are calculated based on the weighted average resource usage by namespace.

2. GCP SKU Resources Query

This query lists all GCP SKU resources related to our tooling (e.g., Mimir, Sentry), including their descriptions and daily prices.

What the Views Display

- GKE Operational Cost: Shows the daily costs for each observability namespace within the GKE footprint over the past 30 days.
- GCP Operational Cost: Breaks down the costs of various GCP services by SKU, such as load balancer data processing and storage snapshots, also for the past 30 days.

SECTION: engineering/infrastructure-platforms/production-engineering/observability/error_budgets.md

We maintain the Error Budgets process that is described in the Engineering Handbook.

Issues are kept in the Observability group tracker with the label of `Category::Error Budgets`.

We maintain the metrics used to generate the Error Budgets and we ensure that the reports are published on time. The reports are available as issues in the Error Budget Reports issue tracker. They are automatically generated by the code in the same project but require manual editing before sending them out.

We advocate for improving the SLOs for Stage Groups and we provide support to help them achieve this. Providing the Stage Groups with data about how their feature categories operate on GitLab.com enables them to make good choices about how to efficiently improve the reliability, availability and performance of their feature categories.

SECTION: engineering/infrastructure-platforms/production-engineering/observability/technical_blueprint.md

The purpose of this document is to detail the goals and guidelines for the Observability team. The focus is on our principles and our goals for the next five years.

Team Principles

Observability for GitLab, not just SaaS Platforms

We are better when we work together.

Although our main mandate is to support GitLab's SaaS Platforms, we actively share knowledge with other teams at GitLab and other customers and build tools and frameworks that work for everyone.

Self-Service, Discoverability and Accessibility

We favor providing repeatable self-service opportunities to those who use our stack over doing the work ourselves, which increases efficiency for everyone.

These opportunities are easy to use and easy to discover for our users.

Internal users can improve the observability of our platform and access observability data freely, through higher level means (dashboards, reports, etc.) and lower-level well-defined interfaces to underlying data.

Improve the product with the tools we create when possible

Not all of what we do belongs in the product, but some of it might!

In line with our Dogfooding philosophy, we go beyond the SaaS context and consider what we can offer upstream to other GitLab customers.

Similarly, we prefer using GitLab's own features and tools when they fit our requirements.

Tool Portability

GitLab's SaaS Infrastructure is complex.

We aim to provide observability solutions that are portable or tool agnostic to efficiently deliver a similar and more cohesive experience across our different platforms.

We implement against Standards

The observability space is a mature and competitive industry, meaning there are usually many different (open source) tools to choose from that have their own benefits and nuances.

We opt for industry accepted standards and implementations, which reinforces our principle of portability.

Data should have one source of truth

We have a lot of data, and duplicating it in multiple different locations becomes expensive.

We maintain a well-defined data stack with a single source of truth for the contained data.

Goals

This is an aspirational list of goals that we would like to achieve in the next five years and should be used as a source of inspiration for project planning and roadmap creation.

Theme 1: Improve how we work with other teams at GitLab

We are faster together. If we improve how we work with other teams at GitLab, we make their jobs and our jobs easier, and we can focus on new and additional problems.

Goals within this theme

- Determine who our internal customers are and make certain that we're meeting their needs by soliciting feedback
- Determine other observability tools and who is using them and who owns them (Sentry, Tableau, etc)
- Dogfood the Monitor:Observability tracing tool for GitLab.com
- Collaborate with Dedicated to help create and integrate Observability Units
- Support the observability components of Cells and other GitLab initiatives
- Work with other teams (SIRT, business analytics, finops, etc) to de-duplicate logging and metrics and define scope of what goes where
- Improve customer experience for error budgets to make it easier for stage groups to focus on the problems in the code, not the problems in the tooling.
- Expand error budgets to additional teams/environments (e.g. staging environment, Dedicated and include Infrastructure teams)
- Expand capacity planning to additional teams/environments (e.g. ops and staging environment, Dedicated, etc.)
- Create an observability Single Pane of Glass (SPOG)

Theme 2: Make it easier for teams to onboard to the observability stack

Adding a new service to the observability stack today is a complicated and not well understood process. Adding new metrics to the source side is also only partially understood. We should create as the right tools and abstractions to make it so that all our customers can self-service onboarding to the system. This will also have the benefit that the system will become easier to maintain.

Goals within this theme

- Provide reference tooling for teams to onboard themselves to the observability stack (helm charts, cloud run configs)
- Create reusable abstractions that can be used for a wide variety of systems within GitLab and are easy to self-service
- Improve our cardinality of metrics and educate teams on the best practices, making it easy to do the right thing and hard to do the wrong thing, and create a feedback loop to help iterate over improvements
- Create well understood patterns to add metrics at the source side (LabKit, etc)

Theme 3: Make the observability stack easier to manage and maintain

In order to work efficiently, we make data driven decisions on the health of our system, improve the ease of use for ourselves and for other teams, and make certain that we're focusing

on self-service.

Goals within this theme

- Set SLOs for availability and correctness of observability tools (metrics, logging, capacity planning, etc)
- Set SLOs for mean time for changes to take effect for metrics and observability deployments
- Use capacity planning to forecast saturation for all aspects of the observability stack
- Improve deployment tooling for observability deployments (helm files, deployment tools, etc)
- Split the runbooks repository into smaller, self contained repositories
- Create a cost effective and easy to manage logging solution
- Push capacity planning as far left as possible (docs on how to use it, live grafana views, etc)
- Create a system to audit metrics, logs and dashboards to delete data that no one uses

Theme 4: Improve our ability to answer FAQ from the past and unknown questions for the future

We've had a number of situations where lack of observability into aspects of our environment has cost us time, money, and availability. We need to constantly iterate to improve how we are able to observe our environment by providing appropriate tools and data to the teams who own the services.

Goals within this theme

- Increased redis key analysis to better understand how we use Redis

We have had multiple incidents and multiple issues where a subset of keys have either been unused and were wasted space or grew substantially without anyone realizing it until it was too late. We also have very little knowledge of which teams own which pieces of data within redis which makes additional analysis even harder. We need to improve our ability to understand how we're using shared data resources so that we improve how we use them.

- Improve database analysis and observability

Database scale is a core problem for us and our engineering teams are dealing with regular and also not so well known problems due to the scale of the database systems we run. They need database observability tech beyond what you'd usually expect to aid database analysis and getting insights into scaling bottlenecks we need to resolve.

- Provide the ability to answer unknown questions with the data we presently have

Theme 5: Improve external customer experiences

- Availability reflects the customer experience
- Create a toolchest for self-managed customers so that they can use the knowledge we have
- Create a capacity planning tool for self-managed customers

SECTION: engineering/infrastructure-platforms/production-engineering/ops/_index.md

Mission

The Ops team is an infrastructure team under Infrastructure Platforms that focuses on improving processes that are vital to the successful operations of GitLab.

Vision

The Ops team's vision is to enable service owners to operate their own services using standardized processes, frameworks, architectures and tools. Some of those processes and tools will be built by the Ops team, but many will be from other Infrastructure teams.

Ownership and Responsibilities

There are two areas that are the Ops team primary focus:

1. Incident Management - Ops is responsible for improving the processes GitLab uses for incident management
2. Disaster Recovery - Ops is responsible for managing our disaster recovery processes with a particular focus on reducing our RTO
3. Patching Processes - Ops is responsible for defining and maintaining the GitLab.com patching process

Getting Assistance

Should you require assistance from the Ops team, please open an issue in the Production engineering tracker and add make sure to add the label ~"team::Ops"

- We also have team handles that ping the full team
- GitLab: @gitlab-org/production-engineering/ops

How We Work - Prioritization

Project Management

The Ops team top level Epic can be found here.
We follow the Infrastructure Platforms Project Management practices as outlined in the Handbook.

OKRs

For Objectives and Key Results, we align with Platforms guidance for creation and structure.

Epics

In addition to the format described in the platforms project management page, these sections may be helpful

markdown

Administrative

<!-- A copy paste section for creating child epics/issues, ensuring that they relate to the current epic and have the correct labels -->

\\\`

/epic [current epic]

/labels ~"group::Production Engineering" ~"Sub-Department::SaaS Platforms" ~"team::Ops"

~"workflow-infra::Triage" ~"Production Engineering::P2"

\\\`

References

<!-- Links to related OKRs, Epics or issues, external resources etc -->

Demos

| Demo Date | Demo Link | Highlights |

|-----|-----|-----|

Decision log

<!-- A collapsible section to aggregate any decisions made along the way. Be sure to include "why" in addition to "what". -->

{{% details summary="Log" %}}

{{% details summary="date" %}}

[decision taken and why]

{{% /details %}}

{{% /details %}}

- Apply any applicable service labels.
- Make sure to give good context for the status and progress of the project in the weekly status update. If the epic is not on-track, please provide a plan for getting back on-track when possible.

Epic Status Updates

Project status is maintained in the description of the top-level epic so that it is visible at a glance. This is auto-generated using the epic issues summary project. You can watch a short demo of this process to see how to use status labels on the epics to make use of this automation.

Issues

Open planned work for our team is located in the Production Engineering project. Issues should be updated whenever significant work occurs. New issues are expected to:

- Link to a related Epic.
- Include the following Labels (update the priority as needed):

text
/labels ~"group::Production Engineering" ~"Sub-Department::SaaS Platforms" ~"team::Ops"
~"workflow-infra::Triage" ~"Production Engineering::P4"

- If there is a service label that is applicable, also apply that.

Processes

Monthly Availability Updates

The Ops team is responsible for ensuring the published Monthly Availability Updates are maintained. This is currently a manual process. Items to update include:

1. Historical Service Level Availability including maintenance windows from the month in the comments

Each of these items should be updated to reflect the most recent month. (Sample MR).

Latest results are on the GitLab.com General SLA Dashboard (internal only)

{{% alert title="Note" color="primary" %}}

Updates must be merged by the 7th day of each month. This is currently a scheduled event on the Reliability Ops Team's Calendar. Contact any member of the team for more details on this process.

{{% /alert %}}

Monthly Review of Incident and Pager Trends

The Ops team coordinates the monthly process to identify incident and pager trends across the engineering organization. This is an async process with the following objectives:

- Identify actions to address issues identified in the Reliability Team Monthly Availability Reports.
- Generate action items based on the review of key metrics for incidents and pages.
- Generate and delegate action items to the relevant teams based on the review process. This includes:
 - Corrective Actions
 - Infradev Issues
 - Reliability Improvement Issues

These efforts are coordinated asynchronously via the GitLab Incident and Pager Trends Monthly Review Agenda

The process is scheduled on the Ops Team Calendar to kick off on the first Tuesday of each month.

The DRI kicking off the process and ensuring its progress is rotated among members of the Ops Team.

All our welcome to participate in the process of identifying trends. EOCs, especially, are encouraged to participate.

Monthly Review of Incident and Pager Trends: How to guide for DRIs

1. Add a new section to the agenda for the current month.
1. Announce that the process is kicking off in infrastructure-lounge and reliability-lounge on Slack and solicit feedback.
1. Week 1: Review the agenda and respond to any questions or comments
1. Week 2: Reply to the announcement thread and solicit additional feedback.
1. Week 2: Review the agenda and respond to any questions or comments
1. Week 3: Review the Identified Trends section of the agenda and coordinate the creation of any required Corrective Actions, Infradev Issues, or Infrastructure Improvement Issues.
1. Week 4: Reply to the announcement thread that the process is coming to a close
1. Week 4: Add an item to the Reliability Leadership Sync Agenda and include a summary of action items created. Please include severity for each item.
1. Week 4: Send a final reply to the announcement thread indicating that the process is closed for the month.

System patching notifications

The Ops team maintains a project patching-notifier that automates the creation of GitLab issues when security problems are detected on our VM based infrastructure.

Details relating to the operation of this notification system can be found in our runbooks.

Continuous Disaster Recovery Testing and Practice

The Ops team creates, manages, and coordinates regular DR Practices (or "Gamedays") to test and measure our Disaster Recovery processes.

Our Disaster Recovery Gameday process can be found here.

Team Members

{{< team-by-manager-slug manager="kkyrala" >}}

Roadmaps

The Production Engineering Ops team maintains roadmaps for our key focus areas:

- View All Roadmaps

Team Impact Overviews

- 2025 Team Impact

Common Links

- Ops Epic List: <https://gitlab.com/groups/gitlab-com/gl-infra/-/epics/1176>
- Issue Board: All issue by priority
- Slack Channel: ginfraops

SECTION: engineering/infrastructure-platforms/production-engineering/ops/dr-practice.md

Purpose

There are many reasons to test and practice our disaster recovery process for GitLab.com.

1. Ensure our processes work as expected.
1. Keep up with changes that could break or complicate our recovery processes.
1. Increase confidence and knowledge of these processes for those who participate in the on-call rotation.
1. Satisfy compliance requirements around validation of disaster recovery scenarios.

Overview

These practices are often referred to as DR gamedays or just gamedays.

Currently, our DR gamedays focus on zonal recovery scenarios, and each one focuses on a specific component.

During a real zonal failure, these gamedays should be capable of being executed in parallel to save time.

mermaid

graph TD

```

A(Assign Gameday) --> B(Perform Gameday)
B --> C{Were there problems?}
C --> |Yes| D(Document problems in the<br>Production Engineering Issue)
C --> |No| E(Record Gameday Measurements)
D --> E
E --> F(Update Confidence Levels)

```

Gameday Assignment

Each quarter, eachPhase 1 component gameday process is assigned to a participating team. This allocation should help us test each process once a quarter.

1. The Ops team will create issues in the Production Engineering issue tracker.
1. This issue is assigned to the next team in order to perform the practice.

Performing the Gameday

1. Create a gameday change issue using the appropriate template.
1. Review the process and consult the Ops team if any part of the process is unclear.
1. Schedule the gameday, seeking out proper review and approvals for the change.
1. Execute the gameday.

It is recommended to work with a reviewer during the process to record problems, timing, and approving merge requests during the process.

Reporting Gameday Process Issues

Compile any feedback from the gameday and update the Production Engineering issue. The Ops team member who created the issues will be the directly responsible individual tasked with incorporating the feedback into future practices.

Recording Gameday Results

Open merge requests to update the timing and execution information from the gameday.

DR Practice Measurements

Updating Confidence Levels

As services change, and processes are improved, the ops team is tasked with managing the confidence level measurements.

SECTION: engineering/infrastructure-platforms/production-engineering/ops/on-call-handover.md

On-call handover

The on-call-handovers project contains issues for each SRE's on-call shift. The outgoing EOC records the activities of their shift using the handover issue template to indicate a handoff and assigns it to the incoming EOC. The `/sre-oncall [handover]` slack command can also be used in the production channel to simplify this process. It will automatically create a new issue and pre-populate some information such as outgoing/incoming EOC handles, open/closed incidents, resolved alerts, etc.

At the end of a SRE's on-call shift, a summary comment of notable events, incidents, etc. should be posted to the Weekly Reliability (SRE) Team Newsletter as a comment. While we do not currently have a prescriptive format for this summary, EOCs can reference this thread for some examples; what specific information is deemed most notable for a given shift will necessarily differ week-to-week, and is left to the discretion of the outgoing EOC.

Once a week the production team has an on-call handover for changing the on-call shift. This meeting is optional but we encourage incoming and outgoing EOCs, Incident Managers, and anyone who is interested in GitLab.com operations to attend.

This meeting will be used to hand over the gitlab.com operations and discuss operational issues related to gitlab.com. There will be a short review of the current on-call report, open issues with the oncall label, user facing errors on the site as well as priorities for the next on-call team.

Team leads are invited and should join in case there are any specific issues related to the service they are responsible for or anything they would like to give attention to.

Before the Meeting

The current on-call will update the document with a report link and items for discussion.

During the Meeting

Discuss items on the "GitLab Oncall Handover" document.

Review 7 day graphs on the current on-call issue report.

Assign all open on-call issues to the next on-call team.

After the Meeting

The current on-call will close the report issue.

SECTION: engineering/infrastructure-platforms/production-engineering/ops/on-call-shadowing.md

The engineer on-call (EOC) shadowing process is designed to provide new engineers with practical, hands-on experience in managing live incidents, responding to alerts, and ensuring system stability. Shadowing allows new team members (Shadows) to observe and gradually take on the responsibilities of an EOC under the guidance of an experienced engineer (EOC Buddy).

This document outlines the key expectations for both the EOC Shadow and EOC Buddy, ensuring a structured approach to learning and support throughout the shadowing process. By clearly defining the roles and responsibilities, we aim to ensure the Shadow gains the necessary skills, knowledge, and confidence to handle real-time incidents effectively, and that the EOC Buddy can provide the appropriate guidance and mentorship.

The goal is to create a collaborative and educational environment that prepares the Shadow to independently manage on-call duties while maintaining system reliability and efficient incident response. This document serves as a guideline to support the successful transition of Shadows into full-fledged on-call engineers, ensuring that they are fully equipped to take on the challenges and responsibilities of the role.

Expectations for the EOC Shadow

The following outlines the expectations for the EOC Shadow during their shadowing period. These steps are intended to help the Shadow gain the necessary experience and confidence before they take on full EOC responsibilities:

1. Join the shadow rotation after completing onboarding: The Shadow should enter the shadowing rotation only after completing the initial onboarding tasks for Site Reliability Engineers (SREs) or their equivalent role. This also includes a discussion with their manager to ensure they are fully prepared to begin shadowing and learning from experienced on-call engineers.
1. Dedicate one day a week to shadowing: The Shadow is expected to commit to shadowing at least one day a week over several weeks. This allows for gradual exposure to EOC duties and gives the Shadow time to build their skill set. As they approach their own on-call shifts, their

involvement should increase, allowing them to take on more significant tasks with greater confidence.

1. Actively participate in incidents and alerts: During the shadowing period, the Shadow should actively engage in incidents and respond to alerts. This includes following along during investigations, observing how decisions are made, and offering assistance where possible. By being hands-on, the Shadow gains practical experience, which is crucial for their development.

1. Check in with the EOC at the beginning of each shift: At the start of every shift, the Shadow should check in with the EOC. This check-in sets expectations for the shift and allows the Shadow to express their readiness to be included in investigations or incident responses. It helps ensure that both the Shadow and the EOC are on the same page regarding tasks and responsibilities for the shift.

1. Join on-call handover discussions: When possible, the Shadow should participate in on-call handover meetings. These discussions provide important context for ongoing issues, incidents that occurred during the previous shift, and tasks to focus on. Attending these meetings is crucial for the Shadow to understand the continuity of operations and prepares them for independent on-call work.

1. Act as the primary responder during the final shadow shift: In the final shadow shift, the Shadow should take on the role of primary responder, managing alerts and incidents as they come in. The EOC will act as secondary support, available to step in within a 5-minute escalation window if the Shadow encounters challenges. This serves as the final test before the Shadow becomes an official part of the on-call rotation.

1. Engage in post-incident discussions and analysis: The Shadow should actively engage in post-incident reviews, where the team reflects on how alerts were handled and what actions were taken. These reviews provide key learning opportunities and help the Shadow gain insights into how incident responses can be improved or handled more efficiently in the future.

1. Participate in the weekly on-call handover meeting: Additionally, it's recommended that the SRE shadow participates in the weekly on-call handover meeting within their time zone. This meeting provides an opportunity to review and discuss the incidents from the previous week, which can be invaluable for staying informed and prepared.

1. Participate in a "gameday" exercise as graduation: Before officially joining the EOC rotation, the Shadow should participate in a "gameday" exercise. This is a simulated incident scenario that allows the Shadow to apply their knowledge and skills in a controlled environment. Successfully completing this exercise serves as a form of graduation, signaling that the Shadow is ready to handle real incidents independently. Check for the gamedays calendar and if one is planned, with the help of the SRE buddy setup one. Check the Gameday calendar, and if one is already scheduled, you can join it. If none is planned, coordinate with the SRE buddy to organize one.

These expectations ensure that the Shadow has the necessary hands-on experience, support, and training to handle the responsibilities of an EOC effectively.

Expectations of the EOC Buddy

The role of the EOC Buddy, or the experienced EOC, is critical in guiding and mentoring new engineers during their shadowing period. The EOC Buddy plays a key role in ensuring the Shadow gains practical insights, understands real-time incident management, and builds the confidence needed for independent on-call responsibilities.

This section outlines the key expectations for the EOC Buddy, focusing on communication, mentorship, and actively involving the Shadow in day-to-day on-call tasks. By following these guidelines, the EOC Buddy helps foster a supportive and educational environment, ensuring the Shadow's successful transition into their future role.

1. Awareness of shadowing: It's important for the EOC Buddy to be aware when they are being shadowed. This should be communicated ahead of time, ensuring that the EOC Buddy knows when someone will be observing and learning from their actions. Proper communication sets the stage for a more productive and collaborative experience.

1. Check in with the Shadow: At the start of each shift, the EOC Buddy should check in with their Shadow. This helps to establish expectations for the shift, including clarifying what tasks or areas the Shadow can assist with. This also ensures that both parties are on the same page from the very beginning.

1. Share thought process during incidents: During the course of an investigation, the EOC Buddy is expected to actively share their thought processes with the Shadow. This includes explaining which dashboards they are monitoring, which runbooks they are following, and the specific commands or actions they are taking. This transparency allows the Shadow to better understand the investigative process and learn in real-time.

1. Keep the Shadow informed: The EOC Buddy should make an effort to tag the Shadow in all relevant incident issues, Slack discussions, and investigations. By doing so, the Shadow remains informed and engaged with ongoing incidents, making it easier for them to follow the case and participate when appropriate.

1. Involve the Shadow in some tasks: The EOC Buddy should encourage the Shadow to take on smaller tasks where appropriate. This could include handling minor alerts or assisting with other requests related to the EOC. These practical tasks provide the Shadow with hands-on experience, which is vital for their learning and development.

1. Offer guidance and clarification: Whenever the Shadow requires clarification or guidance, the EOC Buddy should be available to provide it. Whether it's explaining a particular alert, answering questions, or giving direction on what steps to take next, the EOC Buddy plays a crucial role in supporting the Shadow's learning process.

1. Participate in post-shift reviews: After the shift, the EOC Buddy is encouraged to participate in post-shift reviews with the Shadow. These reviews offer a chance to reflect on the alerts handled and actions taken during the shift. It's an opportunity for the Shadow to ask questions, and for the EOC Buddy to provide feedback, ensuring the experience is as educational as possible.

SECTION: engineering/infrastructure-platforms/production-engineering/ops/sre-onboarding.md

Onboarding Template

SRE onboarding is mostly handled by two issue templates:

1. Machine setup
1. Gather context

These are assigned to the SRE when they start. This will guide them through different areas of the system, starting off with some simple tasks and help both the SRE and the SRE manager through various access issues.

There is a third issue template for oncall onboarding, which should be completed after the first two and will probably take at least 3 months from the start date to complete.

GitLab.com Infrastructure Management

The SRE teams use Terraform and Chef for configuration management of GitLab.com infrastructure.

Terraform

Terraform configuration is currently divided into three environment:

production
staging
ops

There is shared terraform config for both staging and production to keep topology parity between these environments. Instance sizing, fleet sizes and other environment specific configuration is set in variable files for staging, production and ops.

The state for terraform is maintained in object storage, the master branch should always represent the current state of infrastructure. Changes should be merged and applied through CI.

Chef

Chef is a critical part of SRE infrastructure management. Currently it is used for OS patching, applying system level configuration and installing the omnibus package for releases. Here are a few notable cookbooks which will be a good starting-point for new SREs:

cookbook-omnibus-gitlab: This cookbook is responsible for creating a gitlab.rb on every server that has GitLab installed. This config file is used by the omnibus package.

gitlab-cookbooks: This is a collection of cookbooks that are used for GitLab.com.

Releases

Releases candidates are deployed to GitLab.com through auto-deployments that occur throughout the week. For information about how releases at GitLab.com read about the releases process visit the release project documentation.
documentation.

For information about GitLab.com deployments and patches see the following release docs:

release documentation for deployer
post-deployment patch documentation

Where to find things

Repositories

The following repositories are used for GitLab.com infrastructure management. These repository locations are the remotes that the SRE team uses for pushes, issues and MRs. Mirrors are setup in case that GitLab.com is unavailable. Repositories that are necessary for assets, configuration, infrastructure, releases and patch management use <https://ops.gitlab.net> as a remote.

1. terraform: This is the repository that holds all terraform configuration for the GitLab.com staging, production and operations environments. There is a repository mirror on GitLab.com .

1. chef cookbooks: These repositories are the cookbooks used for GitLab.com. Runlists for the fleets are configured in roles. There are repository mirrors for these cookbooks on ops.gitlab.com.

1. chef: This repository contains all role and node attributes for GitLab.com infrastructure. It also has the environment configurations for production, staging and ops for cookbook version pinning.

1. runbooks: This repository contains runbooks, howtos and alert definitions for GitLab.com. Alerts defined in this repository are automatically applied to the monitoring infrastructure when merged to master. For more information see the alert section.
There is a repository mirror on ops.gitlab.net.

Dashboards

It is useful to have the following dashboards bookmarked and easily accessible

- 1. Grafana
- 1. Google Cloud
- 1. System Logs
- 1. Fastly CDN

Cloud Providers

- 1. Google Cloud
- 1. Amazon Web Services

Monitoring tools

- 1. PagerDuty Alerting
- 1. Grafana Performance Monitoring
- 1. Alert Dashboard

Issue Trackers

It is useful to have the following issue trackers bookmarked and easily accessible

- 1. On Call Issues
- 1. Production Incidents Issues
- 1. Change Management Issues

Yubikey

SREs should be using a YubiKey and should not have keys on their laptop. It is recommended to have a spare YubiKey to avoid being locked out of your accounts if you lose your primary key.

Follow the yubikey runbook to set up

Credentials

The following is intended to be a comprehensive list of credentials and access that need to be set up, which are not covered above or elsewhere in the handbook. The list may not be up to date. If something is missing, please add it.

- 1. SSH Key - this is provided to you by the yubikey setup
- 1. GitLab.com account
- 1. GitLab.com admin account
- 1. dev.GitLab.org account
- 1. ops.GitLab.net account
- 1. Chef access
- 1. Cloud Providers

Amazon Web Services

Azure

Digital Ocean

Google Cloud

Slack Channels

Oncall-related channels:

- production
- incident-management
- alerts
- announcements
- dev-escalation
- feedalerts-general
- cloud-provider-alerts

Infrastructure channels:

- srestandup
- infrastructure-lounge
- infra-lounge-social
- infra-read-feed
- gdelivery
- gscalability
- infracapacity-planning
- reliability-lounge
- ansible
- kubernetes
- terraform

Zendesk

Every SRE should register for a "Light Agent" account in ZenDesk. Often times incidents are generated from customer reports, and it's useful to see their submission and the back and forth with support. You can also leave internal notes for support engineers so that they can gather more information for troubleshooting purposes.

Time Off by Deel

We use Time Off by Deel to notify and delegate for planned timeoff. When setting up your integrations with Slack, be sure to run the `/time-off-deel help` command then click Go to Home Tab then in the drop-down select Calendar Sync then Add Calendar to add the team's shared Google Calendar (ID: `gitlab.comoji6dki1frc8g8qq9feuu1jtd0@group.calendar.google.com`) as an "Additional calendars to include?".

Suggested Software Tools

As production engineers we are allowed to utilize a linux workstation. The list below is mostly comprised of macOS tools. You'll need to find the linux equivalent to match the linux distro of your choice.

In addition to the standard tools for interacting with the rest of GitLab, the following tools help when working on production issues.

Required tools

1. Homebrew
1. SSH, properly configured
1. chef, knife, berkshelf
1. kubectl (brew install kubernetes-cli)

Nice to have

1. iTerm (brew install iterm2) or kitty (brew install kitty) (bear in mind that kitty requires more configuration to get it up and running so it's targeted at more advanced users)
1. macOS doesn't source ~/.bashrc file by default, so if you want it to be processed, you need to source it in your profile file (which you might need to create manually). Why to create the rc file at all instead of keeping everything in the profile? some tools default to rc so they will not process the profile at all. There are actually more differences, see: About bashprofile and bashrc on macOS
1. macOS doesn't have bash completion feature by default, to install it: brew install bash-completion and enable it: echo "[-f /usr/local/etc/bashcompletion] && . /usr/local/etc/bashcompletion" >> ~/.bashrc
1. fzf used for fuzzy completion in shell, e.g. history search or filepaths, (brew install fzf + echo "[-f ~/.fzf.bash] && source ~/.fzf.bash" >> ~/.bashrc)
1. the default length of bash history on macOS is 500, to extend the number of entries kept and save the timestamp you can add to your .bashrc for example:

console

```
export HISTFILESIZE=2000000
export HISTSIZE=1000000
export HISTTIMEFORMAT="%d/%m/%y %T "
```

1. helm - "Kubernetes package manager" (brew install kubernetes-helm)
1. minikube (brew install minikube) and virtualbox (<https://www.virtualbox.org/wiki/Downloads>)
1. GCP cli gcloud quickstart macos
1. Digital Ocean cli (brew install doctl)
1. Azure cli (brew install azure-cli)
1. AWS cli (pip3 install awscli --upgrade)
1. A text editor such as Sublime, Textmate, MacVim, or neovim
1. watch (brew install watch)
1. tmux/tmate (brew install tmux tmate)
1. A markdown editor such as macdown (brew install macdown)
1. BitBar with GitLab Plugin
1. To install gnu utils and replace mac utilities use the --with-default-names option.
1. when using gpg, you will be asked for a password. Querying for passwords can be facilitated by different tools, but a fairly standard and widely supported one is pinentry-mac (brew install pinentry-mac). To tell your gpg agent to use it: echo 'pinentry-program /usr/local/bin/pinentry-mac' >> ~/.gnupg/gpg-agent.conf

Brew Files

There are sample brew files in the Infrastructure Project

iOS apps

1. Slack
1. Zoom
1. PagerDuty
1. Working Copy (Optional)

Reference Material

List of relevant reference material that an engineer may need to brush up on

1. Chef
1. Terraform Docs or getting started guide

SECTION: engineering/infrastructure-platforms/production-engineering/ops/sre-oncall-onboarding-buddy.md

Introduction

Engineers on-call (EOC) onboarding buddies play a crucial role in ensuring a positive and effective onboarding experience for new engineers joining the on-call rotation. Being on call can be very stressful, particularly for engineers who are new to the role or unfamiliar with our systems and processes.

That's why it's essential that all new engineers joining the on-call rotation be assigned a buddy who is ready and willing to assist with the onboarding process. These buddies are experienced on-call engineers who can provide guidance and support and share their invaluable knowledge.

The EOC on-call buddy system is designed to:

- Accelerate the on-call onboarding process for new engineers
- Reduce stress and anxiety associated with on-call responsibilities
- Help new engineers gain confidence in their ability to handle incidents
- Ensure that new team members become self-sufficient on-call engineers more quickly
- Promote knowledge sharing and best practices within the team

By pairing new engineers with experienced buddies, we create a supportive environment that allows for faster learning, better incident response, and a more resilient team overall. This system not only benefits the new engineers but also strengthens our entire on-call process by continuously refining and sharing our collective expertise.

Definitions used in this document:

- EOC-Buddy: An experienced on-call engineer who volunteers to mentor and guide new engineers through the on-call onboarding process
- EOC-Shadow: A new engineer who is going through the on-call onboarding process under the guidance of a Buddy

Buddy Responsibilities

The role of a EOC-Buddy is important in helping new engineers become confident and effective on-call responders. Here are the key responsibilities:

- Schedule an initial call with the EOC-Shadow
- Review the on-call runbook project together and explain the general workflow of incident response
- Ensure EOC-Shadow has access to all necessary tools, dashboards, and alerting systems
- Walk through the escalation policy and communication channels used during incidents
- Invite EOC-Shadow into incident Zoom calls during active incidents
- Share personal experiences and lessons learned from past incidents
- Be available as primary respondent during their shadow shifts
- Provide secondary support during EOC-Shadow's first few on-call shifts
- Check in periodically on the progress and offer help

How Can I Become An On-call Onboarding Buddy?

The EOC buddy program operates on a volunteer basis, allowing experienced engineers to contribute to the team's growth based on their willingness and capacity. Please make sure to check the Buddy Responsibilities section.

If you are willing to become a EOC-Buddy, add yourself to @eoc-buddies Slack group via sre-oncall-onboarding-buddy project. This Slack group receives notifications when EOC-Shadows are looking for buddies. When you find a suitable match, respond to the request and notify both your manager and the EOC-Shadow.

Workflow

Being an EOC-Shadow

1. Join the eoc-shadowing Slack channel introduce yourself
 1. Ping the @eoc-buddies Slack group in the channel, specify your timezone and availability and clearly state that you're looking for a buddy
 1. Once a Buddy is assigned, schedule an initial coffee chat with your new EOC-Buddy. Get to know each other and discuss expectations and suitable schedules for sync calls
 1. Join the SRE Shadow rotation schedule in PagerDuty and align your shifts with your EOC-Buddy's on-call shifts.
 1. Schedule weekly 45-minute calls with your EOC-Buddy. These calls should occur after each on-call weekly shift.
- Review and discuss incidents from the past week
1. Begin responding to S3/S4 incidents as primary responder. Your Buddy will provide guidance and support. Document learnings and questions
 1. Continue working through the SRE-oncall onboarding issue, track your progress and completion of required tasks

Being an EOC-Buddy

1. Join the eoc-buddies Slack group
1. When a buddy is requested via the Slack group, review if the timezone is suitable and if you

have the capacity to be assigned to the EOC-Shadow

1. Organize initial coffee chat with your EOC-Shadow

1. Discuss the time for periodic call after each of your weekly shifts. Review the process for the Shadow to be a primary respondent for S3/S4 incidents. Be available as a secondary respondent for each incident

1. Regularly check onboarding progress and provide additional support when needed

FAQs

What if I cannot find a buddy?

Please talk to your manager and ask for help. Your manager will look into available engineers in your timezone and find a suitable buddy.

Can I shadow other engineers and not just my on-call buddy?

Absolutely! It is encouraged that you shadow any EOCs and incidents whenever you feel is right for you. Meeting with other engineers will help you to get a wider knowledge and diverse approach to troubleshooting and incident management.

SECTION: engineering/infrastructure-platforms/production-engineering/ops/roadmaps/_index.md

Overview

This section contains the roadmaps for the Production Engineering Ops team, organized by fiscal year.

Current Roadmaps

- FY26 Roadmaps

SECTION: engineering/infrastructure-platforms/production-engineering/ops/roadmaps/fy26/_index.md

Overview

This section contains the FY26 roadmaps for the Production Engineering Ops team's key focus areas:

- Incident Management
- Disaster Recovery Verification
- Patching & OS Modernization
- Hosted Runners

SECTION: engineering/infrastructure-platforms/production-engineering/ops/roadmaps/fy26/disaster-recovery.md

References

RTO High-Level Overview

Background

Disaster Recovery covers the tools and processes needed to restore GitLab.com to a working state with all customer data in a disaster event. It is a controlled activity that is reported on for compliance purposes. We must have confidence in our ability to recover from a disaster event, and we must be able to communicate that confidence to our customers through consistent, measurable results.

North Star

Disaster Recovery is hands-off and built-in to our system. It is tested in an automated way to validate our ability to recover from disaster events. We use automated testing results to demonstrate our recovery capabilities to our customers. This is consistent for both .com and Dedicated.

Goals for FY26

Push gamedays from process reaction to verification

Right now, gamedays for disaster recovery have mostly been focused on areas in GitLab.com that require manual intervention for zonal degradations. This is important, but there are areas in GitLab.com and the new Cells infrastructure that should be resilient by design, and we should test and verify that design.

This also includes providing feedback for service owners about how improvements could help move from reactive response to a more proactive service implementation.

Push gamedays into production

Gamedays have been targeted on our staging environment (GSTG) to avoid disruptions and to gain confidence. Verification type gamedays that test resilience should be conducted in production to better inform our timing data and to verify that production is ready to handle degradations.

Constant gameday improvement

Continue to execute gamedays regularly. Collect feedback to improve services tested, and the gamedays themselves. Improve automation to help minimize toil and look for opportunities to use automation to perform DR verification. Use gameday measurement to inform RTP/RPO values. Leverage instance management tooling to help disaster recovery related responses be performed quicker and with confidence.

Document and communicate disaster recovery ownership

Update our Ops handbook and processes to clearly outline what the team's goals are in testing and verifying disaster recovery capabilities. This should include how we interface with other teams and provide feedback on improving disaster recovery response and capability.

Exploring DR in Cells

Figure out how to enable Geo for GCP dedicated, and make that a default configuration for new cell creation. Re-configure the existing cell to make sure that Geo is enabled. Configure Backups/snapshots for each Cell's GitLab instance so we can perform data recovery.

SECTION: engineering/infrastructure-platforms/production-engineering/ops/roadmaps/fy26/hosted-runners.md

Table of Contents

- Background
- North Star
- Current Landscape
- Goals for FY26

Background

GitLab-hosted runners (a.k.a hosted runners for GitLab.com) are managed, deployed, and maintained through cross-team collaboration. SRE supports and sometimes leads deployment solutions to ensure the scalability and reliability of the service in Production.

In FY25, SRE supported Runners in the following chronologically-sorted efforts:

1. Assisted Dedicated Runners with the Beta release in AWS
2. Released new runner type offerings: Medium and Large ARM64-based runners
3. Explored PoC - Deploying Hosted Runners using Dedicated Runners architecture
4. Found a solution to the VPC peering limits which prevented the scalability of .com hosted runners
5. Implemented the VPC re-design across all .com runner shards
6. Consolidated .com runner environment deviations in terraform

In FY26, the focus will be on enabling the runners team to better self-service runners operational work.

North Star

Achieving fully automated, hands-off scalability and maintainability for GitLab-hosted runners, ensuring seamless, continuous operations with minimal manual intervention.

Current Landscape

The following sections highlight the shortcomings of the current landscape. While focusing on areas needing the most growth may seem pessimistic, our setup and processes are in a better

position today compared to a year ago. However, we're still far from ideal and have room for improvement.

Knowledge Silos

Access to information about maintaining and scaling Hosted runners is limited. Expertise is demonstrated by a few engineers, and the process is not well-documented.

Scalability Fatigue

The scalability process, both for offering new runner types and for scaling existing ones, is hands-on, demanding, prone to human error, repetitive, and boring.

Cost Efficiency

The cost efficiency for some shards is far from ideal. While insights to this data are available for the FinOps team, they're not empowered to take action or make decisions based on this data. Additionally, we're not investing in exploring alternative deployment methods or understanding the current CPU usage to minimize the cost of hosting .com runners.

Goals for FY26

Effort Level Key:

- LF: Low Effort
- MF: Medium Effort
- TF: Tremendous Effort

Knowledge Transparency

Objectives:

1. Improve the quality of deployment and scalability docs (MF)
1. Create discoverable, easy-to-reach information (LF)
1. Convert docs to CR template for better cross-team transparency (LF)
1. Coordinate with Runners team to transfer Runbooks ownership (MF)

Related Links:

1. Hosted Runners - improve Runbooks

Scalability Fatigue

Objectives:

Automate scaling the existing shards process:

1. Author epic/issue in the runner's tracker to (LF):
 1. Improve existing tools (deployer and GRIT)
 1. Consider embedding the process in a pipeline
 1. Research alternative deployment solutions (e.g., revisit runner-managers in k8s)

Related Links:

1. Runners - automate the scaling process
1. Future iterations for Runner deploy improvements

Cost Efficiency

Objectives:

1. Work with FinOps to determine objectives (LF)
1. Achieve reasonable cost efficiency for all existing shards (MF)
1. Explore more cost-efficient hosting methods (MF)
1. Optimize resources usage (TF)

Related Links:

1. Investigate FinOps Cloud Efficiency vs Grafana Idle Efficiency
1. Reduce autoscaling parameters for larger Linux runners due to high compute inefficiency

SECTION: engineering/infrastructure-platforms/production-engineering/ops/roadmaps/fy26/incident-management.md

Prior art and links

1. Incident Management Overview from 2024-02
1. Overhauling Incident Management from 2024-02

Background

Incident Management includes:

1. Incident Management Processes
1. Staffing for Incident Response
1. Tooling

In this document, we are setting out the FY26 goals for these areas.

North star

Teams can support the services they build, including taking ownership of on-call for them. The Ops team offers a platform that enables teams to easily find help when they need it and ensures they adhere to standard processes.

Goals for FY26

Incident management processes

1. Standard, well-defined incident workflows across all incident types

- 1. Clear role definitions and responsibilities
- 1. Uniform processes for SIRT and regular incidents
- 1. Effective handling of concurrent incidents
- 1. High-quality, maintained runbooks owned by service teams
- 1. Comprehensive, measurable IMOC program
 - 1. Clear performance expectations and feedback
 - 1. Standardized training and documentation
- 1. Data-driven incident management improvements through robust reporting

Tooling

- 1. Implement incident.io for GitLab.com and Dedicated
- 1. Modify post-incident process to incorporate incident.io
- 1. Ensure accurate incident data collection occurs on all incidents

Staffing for incident response

- 1. Focused EOC role with clear scope and responsibilities
- 1. Reliable incident coordinator role with consistent performance
- 1. Well-defined infra leader role with clear regional coverage
- 1. Engineering-wide on-call program

Each of these goals represents a desired end state. The specific implementation plans, including how we'll achieve these goals, what tools we'll use, and what processes we'll follow, will be determined separately based on the prioritization of these goals.

We need to have team members to staff incident response and they need to be equipped to execute on the workflows.

The EOC role currently includes many non-incident related tasks. This role needs to be refined and focused. The EOC should not be used for activities that should be fed into specific teams.

The IM role needs support to become a reliable incident coordinator. With a large membership pool and a long time between participation on incidents, it is easy for best practices to be forgotten. For the most critical incidents, we often default to relying on specific team members to carry us through. If we want this to be the standard, we should be specific about this.

The Infra Leader role thankfully has not needed to be used much in the past few months. However, we needed clearer guidance about what this role entails, who is part of this rotation, and how handoffs between regions are handled.

EOC in Engineering

There is a project to establish wider on-call responsibilities in Engineering. Please see FY26 Directional Theme - On-Call Enhancements across Engineering for details.

SECTION: engineering/infrastructure-platforms/production-

Background

In FY25, the Ops team took ownership over the patching processes that are applied to the supporting infrastructure of GitLab.com. An overview of the areas we currently have applied focus:

1. Establishing runbooks for VM-based patching operations
2. Automating notifications to service owners for required patches and reboots
3. Creating an automation framework for patching our VM fleet

North star

All deployed software represents a potential security risk: we aim to ensure that all vulnerabilities impacting the supporting infrastructure of our SaaS products are able to be detected and resolved in an automated fashion.

Goal proposals for FY26

1. Stop deploying new instances on Ubuntu 20.04
2. Document services that must run on virtual machines
3. Address patch gaps in non-VM infrastructure workloads
4. Align processes with Dedicated/Cells
5. Establish update cadence for shared runner COS images
6. Move runners-manager instances to Kubernetes

Additional details

Stop deploying Ubuntu 20.04

1. This OS reaches end of life in April 2025
2. Many instance types still need to run on VMs, with architecture migration unlikely before EOL
3. Production Engineering completed most Ubuntu 22.04 support work in FY25 Q4
4. We need additional testing with service owners before fully deprecating Ubuntu 20.04

Document services that must run on virtual machines

1. Conversations come up regularly that discuss migrating a service into Kubernetes / replacing it with a cloud native offering. We could collect this knowledge in a handbook page (or similar) so that we understand why it is prohibitive to change the service's architecture, or any potential replacements and the prerequisites required before they become feasible.
2. Ideally this can help in determining feasible options for deploying the services on infrastructure that is easier to maintain, and facilitate future planning.

Address patch gaps in non-VM infrastructure workloads

1. Examine infrastructure workloads such as: Kubernetes pods, CloudRun containers, Cloudflare workers, etc.

2. Determine whether or not existing workflows are satisfactorily keeping these up to date.
3. Add automation to address any shortcomings in the discovery and update processes.

Establish update cadence for shared runner COS images

1. We will implement a standardized update schedule for Container Optimized OS on our CI job VMs to ensure consistent security patching

Align processes with Dedicated/Cells

1. We should evaluate the existing state of security patching in the Cells/Dedicated deployments, and identify any overlap with the current GitLab.com deployment.
2. Identify and address any deficiencies in the patching processes
3. Look for potential efficiency gains by combining processes, removing redundancies where possible

SECTION: engineering/infrastructure-platforms/production-engineering/project-management/index.md

Project Management

The majority of our project management process is described at the Infrastructure Platforms level and is shared between all Infrastructure Platform teams.
Please read this first.

This page describes the additions to the process described on the Platforms page.

The single source of truth for all work is Infrastructure Platforms - Production Engineering Projects Epic.

We often refer to this as our group top-level epic. Each team in the group has it's own top-level epic.

Epics that are added as children to a top-level epic are used to describe projects that the team undertakes.

Project status is maintained in the description of the top-level epic so that it is visible at a glance. This is auto-generated using the epic issues summary project.
You can watch a short demo of this process to see how to use status labels on the epics to make use of this automation.

Teams are in different stages of maturity and may not yet perform all of these project management tasks.

Project Ownership

Each project has an owner who is responsible for delivering the project.

The owner needs to:

1. Update the status block in the epic description each week.
1. Work with others to move project issues through the boards.

Project Workflow

Workflow labels for issues also apply to epics.

Preparation

In this stage, projects are rough ideas on the Opportunity Backlog, that are formed into proposals for discussion on the Roadmap. They are developed into full projects that are then ready to be picked up by the team. We keep track of these projects in our roadmaps.

Opportunity Backlog

-

When we have ideas for projects but we aren't certain that they are the right thing to do, we use the Opportunity Backlog.

There is one Opportunity Backlog epic per roadmap.

Ideas are documented in issues or epics and attached to the relevant Opportunity Backlog epic. These issues and epics can have very little detail and have the workflow-infra::Triage label.

The Product Manager and Engineering Managers regularly review the Opportunity Backlog. In a review session, they will note on the issue or epic if:

- An idea should be built into a proposal (the EM will then work with an Engineer to build a proposal). The engineers assigns themselves while working out the proposal and when done applies ~"workflow-infra::Proposal".
- An idea was considered but is not high enough priority to act on at this time. The issue remains labeled ~workflow-infra::Triage.
- An idea is not relevant and the issue will be redirected to another team or closed

Roadmaps

-

-

-

The EM and PM manage these roadmaps together. The PM ensures that the Roadmap reflects the priority of work required from this team. The EM ensures that there is a backlog of work ready for their team to pick up (See Ready For Development below).

The roadmap epics should have the workflow label of workflow-infra::Proposal so that it shows in the correct section of the top-level epic for the Grand Review.

Any project that is being prepared should be a child of a roadmap epic. These epics should have the workflow labels of either workflow-infra::Triage, workflow-infra::Proposal or workflow-

infra::Ready.

A Roadmap epic has three sections: Now, Next and Later. These list, in priority order, what the team is working on, what they will do next, and what they might address at a later date.

Now

-

For the 'Now' section, the common pattern is to link to the team's top-level epic so we have a consistent SSoT for the work in progress.

Projects in this section are in progress.

Next

-

The 'Next' section contains projects that are the highest priority projects on the roadmap and are ready for development.

During review, the Engineering Manager or Product Manager select the highest priority projects to be added to this list per roadmap.

Later

-

-

This section contains projects that are either ready for development or still proposals.

During review, the Engineering Manager or Product Manager will indicate if more information is required to accept a proposal.

A Roadmap epic can have sub-epics that are also roadmaps. For example, the Practices team will have a sub-epic for Redis and the Observability team will have a sub-epic for Capacity Planning. These help to keep the narrative for each category in a single place. We will also implement automation for the flow of information between levels.

Proposal

We want to strike a balance between having a plan and over-planning.

The purpose of the Proposal state is to answer questions to clarify the goal of the project and broadly define the work required to achieve the goal.

When preparing a proposal, confirm with your Engineering Manager as to how much effort should be applied to this proposal. The Engineering Manager will be able to provide more context as to the current level of detail required.

Ready for Development

Epics that are in the workflow state of proposal will be reviewed by the team to see if they can be advanced to ready for development.

The Engineering Manager asks an Engineer to review and decide whether more detail needs to be worked out upfront or we can advance the epic into ready for development.

For an epic to be ready, the following information must be present on the epic:

1. DRI (assignee) (only required when in 'Next' section)
2. Participants (only required when in 'Next' section)
3. Context - why are we working on this (problem, background)
4. Goals or Impact - what do we expect this project to accomplish
5. Dependencies and Stakeholders - who is relying on this work, who should be informed, who should be consulted
6. Phases of Delivery
7. Project Management Arrangements - expectations of the DRI, EM and Participants, and specific status reporting requirements, if sub-epics or milestones are to be used
8. Status block - should be the final heading in the description with the format Status yyyy-mm-dd.
 1. This enables others who are interested in the epic to see the latest status without having to read through all comments or issues attached to the epic.
 1. This heading is used to auto-generate the status information on the top-level epic.
 1. If the epic has no child-epics and a mermaid block is added, this script that runs on a pipeline will automatically generate and include an issue relationship diagram in this section. An example can be seen in this epic.
9. Start date - set to the expected start date, and updated to be the actual start date when the project is started.
10. End date - set to the expected end date. This should be seen as a target, and this target is re-evaluated every few weeks while the project is in progress. The date that a project actually ended is taken from the date that the epic was closed.

Other items that can be included are:

1. Exit criteria for the specific goals of the project.
 1. These are created as issues with the exit criterion label in the epic and are linked in the description.
 1. We create these at the start of the project to allow us to